

SWASTHYASHIELD EDGE

SIH26181 - 10-Day Build Handoff

NORTH STAR

A privacy-first wearable + mobile companion that learns a user-specific baseline, fuses physiological and disaster context, detects risk locally, explains why it is concerned, and continues operating when the internet is unavailable.

Purpose: hand this PDF directly to Codex, Claude, or a development team and build the prototype without inventing missing product decisions.

Scope	Decision
Problem statement	SIH26181 - Qualcomm Inc - Hardware - MedTech / BioTech / HealthTech
Prototype target	ESP32 sensor node + Flutter Android app + on-device TFLite/LiteRT anomaly model
Core demo	Heat-stress-like anomaly detected locally, airplane mode on, alerts still work
Cloud dependency	None for core monitoring or inference
Build window	10 days - freeze scope aggressively
Safety framing	Early-warning / wellness prototype only; never diagnose disease or claim clinical-grade accuracy

Prepared: 1 September 2026

Web-verified references are listed at the end. Problem wording/ID was cross-checked from current community mirrors; re-check the official SIH portal before final submission.

How to use this handoff

Read Sections 1-4 first. Then give Section 18 to your coding agent as the master prompt.

1. Build objective

Build the smallest prototype that makes the judge believe three things: (1) the system is truly personalized, (2) the intelligence runs on-device and survives a network outage, and (3) the system is useful specifically during Indian disaster conditions such as heat waves, floods and pollution events.

- [] ESP32 streams live sensor telemetry over BLE to an Android phone.
- [] App shows HR, SpO2, temperature/environment, activity/fall state, signal quality and connectivity.
- [] App creates a personal baseline and displays deviation from baseline, not only fixed thresholds.
- [] A TFLite/LiteRT model runs locally and emits an anomaly score from a rolling feature window.
- [] Risk engine fuses anomaly score + heat/environment + disaster context into a 0-100 risk score.
- [] Airplane mode can be enabled and live monitoring + local inference + local alerts continue.
- [] NDMA SACHET disaster adapter caches the latest official alert data when network is available.
- [] Emergency packet is generated locally and can be shared when connectivity exists.
- [] Demo/replay mode exists so the final pitch never depends on unreliable real-time physiology.
- [] No screen claims to diagnose heat stroke, heart attack, asthma, or another disease.

10-DAY RULE
Do not build login, cloud dashboards, doctor booking, a general chatbot, microservices, a complex backend, or custom hardware PCB before the core offline demo is stable.

Layer	Choice	Reason
Sensor MCU	ESP32 DevKit	BLE + inexpensive + fast Arduino ecosystem
Mobile	Flutter, Android-first	Fast UI iteration; BLE/TFLite ecosystem; single repo
On-device ML	TFLite/LiteRT	Straightforward Android deployment; Qualcomm AI Hub can optimize/profile
Local storage	SQLite/Drift + secure key/profile storage	Offline-first and inspectable
Disaster feed	NDMA SACHET RSS/CAP adapter	India-specific, official disaster alert system
Optional environment	BME280 + cached public weather/air data	Local heat signal even without network

Product strategy

What makes the project different from a generic smartwatch app.

2. The standout idea

ONE-LINE PITCH

SwasthyaShield is not a fitness tracker. It learns what is normal for your body, combines that with local disaster conditions, and detects unusual risk patterns on the phone itself - even when the network goes down.

Differentiator	What the judge sees	Why it matters
Personal baseline	"Your normal HR" and deviation trend	Avoids a one-size-fits-all threshold demo
Edge inference	Airplane mode ON; anomaly score still changes	Directly demonstrates privacy + disaster resilience
Disaster fusion	NDMA alert / heat context changes risk mode	India-specific rather than generic wearable monitoring

- Outdoor workers and delivery personnel exposed to extreme heat.
 - Elderly people who may not recognize a deteriorating pattern early.
 - People in areas where connectivity or healthcare access can be disrupted by floods/cyclones.
 - Citizens exposed to severe pollution where personal physiological response matters more than a generic city-level alert.
-
- Clinical diagnosis, medical-device certification, medication recommendations, or treatment plans.
 - Continuous cloud upload of raw health data.
 - Replacing doctors, ambulances, or government emergency systems.
 - Training a large language model or using an online LLM for the core safety loop.

Judge-safe language

Say "early-warning prototype", "risk pattern", "unusual deviation", "wellness guidance" and "requires clinical validation". Do not say "we predict heart attacks" or "our sensor diagnoses heat stroke".

User journey and demo scenario

Build around one memorable story: a developing heat-risk pattern.

3. Primary user flow



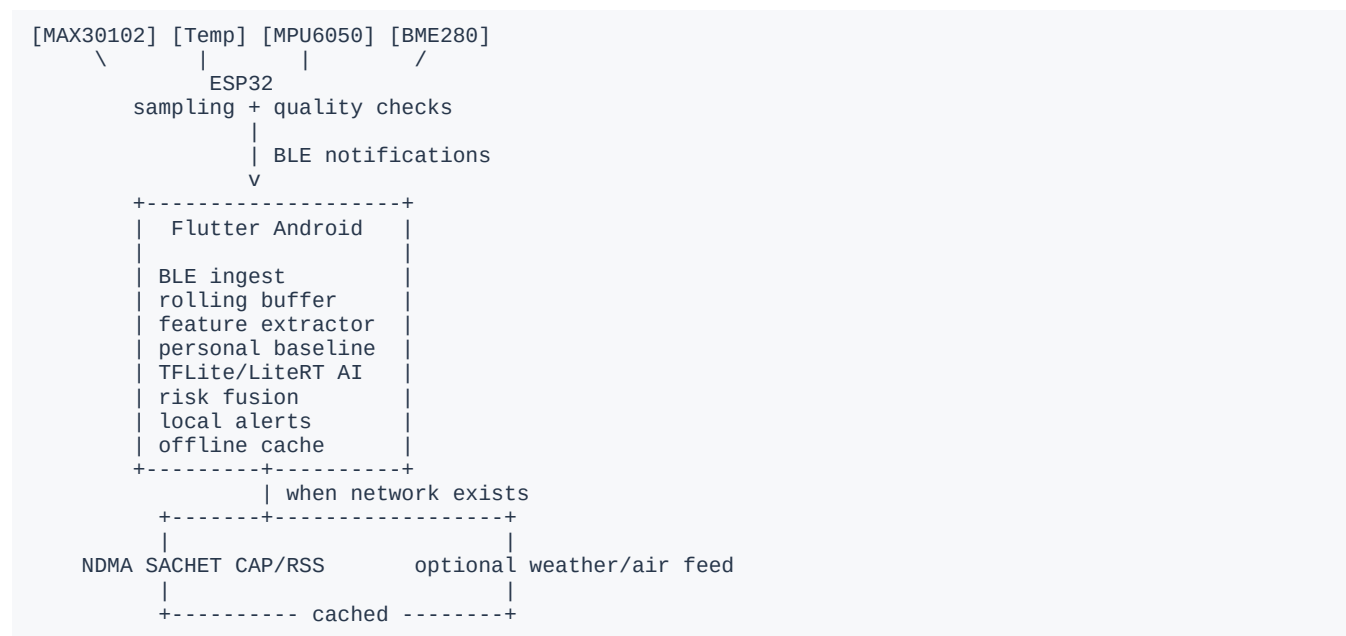
1. Open the app and show live readings in NORMAL state with a personal baseline already learned.
2. Start a prerecorded/replay scenario (or safely alter the demo sensor stream) where HR rises, temperature/heat context rises, and SpO2 trends down slightly.
3. Show "Why this alert?" with the top contributors: HR above baseline, high heat index, prolonged exposure, anomaly score.
4. Enable airplane mode. Keep the sensor stream and model running. Point to the OFFLINE badge.
5. Risk moves to WARNING/CRITICAL from local inference. The app produces offline guidance and prepares an emergency packet.
6. Restore network and show cached/updated disaster context and share/export capability.

Scenario	Expected output
Normal	Risk < 30; no noisy notification
Heat exposure	Risk rises because heat context + physiological deviation agree
Bad sensor contact	Signal-quality warning, NOT a health warning
Fall	IMU spike + immobility -> confirmation screen -> escalation if unanswered
Pollution/disaster context	Context banner + increased caution; no fake diagnosis

System architecture

The core safety loop has no server dependency.

4. High-level architecture



- Raw sensor data stays on the phone by default. Upload nothing for the core prototype.
- External feeds are context only. If they fail, physiological monitoring remains available.
- Every external provider sits behind an interface so a mock/replay provider can replace it during judging.
- The local risk engine must never block on network calls.
- Persist only what the demo needs: recent summaries, baseline, alert history and cached context.

Module	Responsibility
SensorNode	Read sensors, validate signal quality, stream timestamped packets
BleTelemetryService	Scan/connect/reconnect, parse packets, expose stream
RollingWindowBuffer	Keep the latest 30-60 s of valid samples
FeatureExtractor	Mean/trend/variance/baseline deviations and IMU features
EdgeInferenceService	Run local TFLite/LiteRT model; return anomaly score
RiskEngine	Fuse model score with environmental/disaster context
AlertCoordinator	State machine, cooldowns, acknowledgement, escalation
DisasterContextService	Fetch/cache/parse NDMA SACHET data
LocalRepository	Baseline, summaries, alert log, cached CAP data

Hardware plan

Use a breadboard or compact perfboard. Reliability beats cosmetic complexity.

5. Bill of materials

Part	Purpose	Priority	Notes
ESP32 DevKit V1	BLE sensor hub	MUST	Use USB power/power bank for demo
MAX30102	Heart-rate + SpO2 demo signal	MUST	Finger placement is easier than wrist for stable demo
MPU6050	Motion + fall/activity	MUST	I2C
BME280	Ambient temp + humidity	MUST	Compute heat-index-like environmental load
MAX30205 or available temp sensor	Body/contact temperature signal	SHOULD	If unavailable, mark value simulated in replay mode
Push button	Manual SOS/acknowledge	SHOULD	Simple GPIO
Buzzer/vibration motor	Local haptic/audible warning	OPTIONAL	Do not risk power issues late
OLED	Standalone node status	OPTIONAL	Only if core flow is already stable
PMS5003	Local PM2.5/PM10	STRETCH	Bulky; skip unless team already has it

Signal	ESP32	Devices
3.3 V	3V3	MAX30102 / MPU6050 / BME280 / MAX30205 if breakout supports it
GND	GND	All sensors
SDA	GPIO 21	I2C bus
SCL	GPIO 22	I2C bus
SOS button	GPIO 27	Button to GND with internal pull-up

Hardware safety

Do not power unknown breakout boards from 5 V unless their board explicitly supports it. Do not build a LiPo charging circuit at the last minute; a USB power bank is safer for a hackathon prototype.

A bad optical signal must become a "reposition finger / low signal quality" state, not a medical alert. Treat MAX30102 output as prototype telemetry, not clinical-grade measurement.

ESP32 firmware contract

Keep firmware boring, observable and deterministic.

6. Sampling and BLE protocol

Signal	Suggested rate	Transmit
HR / SpO2 derived values	1 Hz	1 Hz summary
Temperature	1 Hz	1 Hz
Ambient T/RH	0.2-1 Hz	1 Hz latest value
Accelerometer	25-50 Hz	Send 1 Hz derived features or compact bursts
Signal quality	1 Hz	0-100 quality score

Use one custom notify characteristic and newline-delimited JSON. Negotiate a larger MTU if available; if not, keep packets compact. The mobile parser must tolerate dropped packets.

```
{"v":1,"ts":1725139200,"hr":78.4,"spo2":98.0,"bt":36.7,
"at":38.2,"rh":71.0,"ax":0.03,"ay":0.12,"az":0.98,
"q":92,"sos":0}
```

```
setup():
  initSerial(); initI2C(); initSensors(); initBLE();

loop():
  sampleFastIMU();
  updateOpticalSensor();
  if (1 second elapsed):
    values = deriveVitalsAndEnvironment();
    quality = estimateSignalQuality(values);
    packet = buildTelemetry(values, quality, buttonState);
    bleNotify(packet);
    logCompact(packet);
  handleBLEReconnect();
```

- [] Runs continuously for 30 minutes without crash.
- [] Reconnects after phone Bluetooth is toggled off/on.
- [] Packet sequence remains parseable if one notification is lost.
- [] Low-quality optical contact is detectable and visible in payload.
- [] Replay/simulation firmware mode can produce Normal -> Warning transitions on command.

Flutter app structure

Do not let UI code contain BLE, ML, parsing or risk logic.

7. Repository structure

```
swasthyashield/  
  mobile/  
    lib/  
      app.dart  
      core/  
        models/  
        storage/  
        utils/  
      features/  
        onboarding/  
        device_pairing/  
        monitoring/  
        baseline/  
        alerts/  
        disaster_context/  
        emergency/  
        settings/  
      services/  
        ble_telemetry_service.dart  
        feature_extractor.dart  
        edge_inference_service.dart  
        risk_engine.dart  
        ndma_sachet_service.dart  
        notification_service.dart  
      assets/models/health_anomaly.tflite  
      assets/demo/*.json  
      firmware/esp32_health_node/  
      ml/  
        train_anomaly_model.py  
        export_tflite.py  
        evaluate.py  
      docs/  
        sample_cap/  
        demo_script.md
```

Need	Typical package / approach
BLE	flutter_blue_plus or equivalent maintained BLE package
TFLite	tflite_flutter / LiteRT-compatible Android runtime
State management	Riverpod OR Provider - choose one, not both
Local DB	Drift/SQLite for history + local summaries
Secrets/profile	flutter_secure_storage
Location	geolocator
HTTP/XML	http + xml
Local notifications	flutter_local_notifications
Charts	fl_chart or simple CustomPainter

Local data and privacy model

Privacy should be visible in the product, not only mentioned in slides.

8. Local data model

Entity	Key fields	Retention for prototype
UserBaseline	hrMean/hrStd, spo2Mean/spo2Std, tempMean/tempStd, createdAt	Persist until reset
TelemetrySummary	minute, hrAvg, spo2Avg, tempAvg, activity, quality	Last 24 h or demo session
RiskEvent	time, level, score, reasons[], acknowledged	Persist for demo history
DisasterContext	provider, type, severity, area, expires, rawId	Latest cached items only
EmergencyPacket	time, latest vitals, trend, reasons, location if consented	Generated on demand

- Core functionality must work with no account and no cloud login.
- Store only derived summaries by default; raw high-frequency IMU/PPG is optional and off by default.
- Keep profile values and local encryption keys in secure platform storage.
- Location is requested only for disaster context/emergency sharing and can be denied.
- Emergency sharing must show exactly what data will leave the device before the user confirms, except an explicitly designed SOS flow.
- Add a "Delete local health data" button that wipes baseline/history.

YOUR DATA STAYS HERE

- Live vitals: processed on device
- AI inference: on device
- Personal baseline: stored locally
- Cloud health upload: OFF

External requests: disaster/weather context only.
No raw health stream is sent with those requests.

Do not overclaim compliance

You can say the architecture is privacy-preserving and designed around data minimization. Do not claim medical-device approval or full legal compliance based on a 10-day prototype.

On-device AI plan

Use AI for anomaly detection; use explicit rules for safety gating and explanations.

9. Model design

The quickest credible approach is a tiny autoencoder that learns the shape of "normal" feature windows. Personalization happens by normalizing input features against the user baseline before inference. The model never outputs a diagnosis; it outputs an anomaly score.

```
[
  hr_z,                # HR deviation from personal baseline
  hr_trend,            # slope over last 30 s
  spo2_z,              # SpO2 deviation from baseline
  spo2_trend,
  body_temp_z,
  ambient_temp_norm,
  humidity_norm,
  heat_load_norm,
  accel_magnitude_mean,
  accel_magnitude_std,
  activity_level,
  signal_quality
] -> 12 float32 values
```

```
Input(12) -> Dense(16, relu) -> Dense(8, relu) -> Dense(4, relu)
          -> Dense(8, relu) -> Dense(16, relu) -> Dense(12, linear)

anomaly_mse = mean((input - reconstruction)^2)
threshold = percentile(normal_validation_mse, 95)
anomaly_score = clamp(scale(anomaly_mse, threshold), 0, 1)
```

1. Collect a few minutes of your own normal prototype sensor data and save window features.
2. Add recorded/replay normal sessions from multiple teammates if possible.
3. Train the autoencoder only on normal windows; do not pretend synthetic abnormal data is clinical truth.
4. Use replay scenarios to evaluate whether combined risk logic reacts in the expected direction.
5. Export to .tflite and verify local inference with a fixed test vector.
6. Optionally compile/profile the model on Qualcomm AI Hub and include measured latency/memory from a real target profile.

Fallback if ML is unstable

Keep the same EdgeInferenceService interface. Return a deterministic anomaly score from baseline deviations while the UI/demo is completed. Swap in the .tflite model once validated. Do not let model integration block the whole project.

Risk engine and explainability

The model is one input, not the sole safety decision maker.

10. Prototype risk fusion

Use explicit, tunable weights for the demo. These are engineering defaults for a prototype, not clinical thresholds. Put all weights in one config file so you can tune the demo without touching UI code.

```
anomalyComponent = anomalyScore
heatComponent    = normalizedHeatLoad * exposureFactor
hazardComponent  = disasterSeverityNormalized
qualityGate      = signalQuality < 0.45 ? 0.0 : 1.0

risk01 = qualityGate * (
    0.55 * anomalyComponent +
    0.30 * heatComponent +
    0.15 * hazardComponent
)
risk = round(100 * clamp(risk01, 0, 1))
```

Level	Prototype score	Behavior
NORMAL	0-29	Dashboard only; no notification
WATCH	30-54	Subtle banner; show top reason
WARNING	55-74	Local notification + advice + recheck timer
CRITICAL	75-100	Full-screen alert + acknowledge countdown + emergency packet

- Do not change levels on one noisy sample; require sustained conditions (for example 3 consecutive windows).
- Use a lower threshold to remain in a level than to enter it, reducing alert flapping.
- Suppress repeat notifications during a cooldown unless the level increases.
- If signal quality is poor, show a sensor-quality message and freeze/discount the health risk calculation.

```
RiskExplanation {
  risk: 72,
  level: "warning",
  reasons: [
    {key:"hr_deviation", label:"Heart rate is above your baseline", weight:0.31},
    {key:"heat_load", label:"High heat and humidity exposure", weight:0.24},
    {key:"anomaly", label:"Your recent pattern is unusual", weight:0.17}
  ]
}
```

Disaster and environmental context

Integrate official alerts, but cache them so the app degrades gracefully offline.

11. NDMA SACHET integration

NDMA SACHET is India's CAP-based integrated alert system and publishes disaster alerts through multiple channels, including an India CAP RSS feed. The official integration guide requires ETag-based client caching: store the returned XML + ETag, send If-None-Match on subsequent requests, use cached XML on HTTP 304, and replace cache on HTTP 200.

```
abstract class DisasterContextProvider {
    Future<List<DisasterAlert>> refresh({Position? position});
    Future<List<DisasterAlert>> cached();
}

class DisasterAlert {
    String id; String event; String severity; String area;
    DateTime effective; DateTime? expires;
    String? instruction; String source;
}
```

```
if online:
    GET feed / alert resource with If-None-Match: cachedETag
    200 -> parse -> save XML + new ETag -> update context
    304 -> use cached XML
    failure -> use cached XML and mark STALE
else:
    use cached XML and mark OFFLINE

IMPORTANT: monitoring + AI never wait for this request.
```

Context	MVP source	Offline behavior
Heat	BME280 ambient T/RH + simple local heat-load calculation	Fully local
Disaster alert	NDMA SACHET cached CAP/RSS	Use most recent unexpired cache
Pollution	Optional feed or PMS5003 stretch hardware	Show last update time / simulated demo event if unavailable
Location	Phone GPS with consent	Last known location; core health loop works without it

Important

Do not scrape random news to create "disaster intelligence". Use an official alert adapter or a clearly labeled replay payload. The judge should be able to tell real data from simulated demo data.

Alerts, escalation and emergency packet

Make escalation smarter than a red SOS button.

12. Alert ladder

Stage	Trigger	UX
0 Normal	No sustained anomaly	Quiet monitoring
1 Watch	Mild sustained deviation	Inline explanation; no panic
2 Warning	Sustained higher risk	Notification, simple action, 10-min recheck
3 Critical	High sustained risk OR fall + non-response	Full-screen acknowledge flow
4 Escalation	Critical not acknowledged / explicit SOS	Generate/share emergency packet

```
{
  "generated_at": "2026-09-01T14:37:00+05:30",
  "risk_level": "critical",
  "risk_score": 84,
  "reasons": ["unusual_pattern", "high_heat_load", "fall_non_response"],
  "latest": {"hr":112,"spo2":95,"temp":37.5,"quality":0.91},
  "trend_5m": {"hr":"rising","spo2":"falling"},
  "disaster_context": {"event":"Heat Wave","severity":"..."},
  "location": {"included": true, "lat": "user-consented", "lon": "..."},
  "disclaimer": "Prototype wellness alert; not a medical diagnosis."
}
```

- Local full-screen alert and phone notification.
- Vibration/buzzer if hardware supports it.
- Emergency packet remains on device and can be shown to another person.
- Optional SMS compose/dialer handoff can be attempted; do not promise internet-based notification in airplane mode.

Possible fall detected

Are you okay? 20-second countdown
[I'M OK] [NEED HELP]

No response -> escalate locally and prepare packet.

UI specification

The judge should understand status in under five seconds.

13. Required screens

Screen	Must show
1. Welcome / privacy	No account required; data stays on device; permissions
2. Pair device	BLE device, connection state, signal/last packet
3. Baseline setup	Calibration progress; "demo quick baseline" option
4. Home dashboard	Risk score, level, online/offline, disaster mode, latest vitals
5. Why this alert?	Top 3 contributing signals + trend
6. Disaster context	Event, severity, area, timestamp, cached/stale badge
7. Alert history	Timeline of watch/warning/critical events
8. Emergency packet	Readable summary + share/export
9. Privacy/settings	Delete data, replay mode, model info, local/cloud status

SwasthyaShield

OFFLINE / ONLINE

PERSONAL RISK

72 / 100

WARNING

"3 reasons detected"

Heart rate

111

+2.1 sigma from your baseline

SpO2

95

-1.4 sigma

Body temp

37.4

rising

Heat load

HIGH

38 C / 71% RH

Signal quality

GOOD

[Why this alert?]

[I am okay]

Disaster context: Heat alert (cached 18 min ago)

AI: ON DEVICE

Health cloud upload: OFF

- Always show last-updated timestamp for external context.
- Use an OFFLINE badge that visibly does not disable the AI card.
- Show signal quality next to vitals to demonstrate sensor awareness.
- Keep one large risk score, not 12 competing gauges.
- A "Why?" button is more impressive than a black-box AI badge.

Implementation contracts

Give these interfaces to the coding agent so components can be built in parallel.

14. Core Dart data contracts

```
class TelemetrySample {
  final DateTime ts;
  final double? hr, spo2, bodyTemp, ambientTemp, humidity;
  final double ax, ay, az;
  final double quality;
  final bool sos;
}

class FeatureWindow {
  final Float32List values; // exactly 12 ordered features
  final DateTime endTime;
}

class InferenceResult {
  final double anomalyScore; // 0..1
  final int inferenceMs;
}

class RiskAssessment {
  final int score; // 0..100
  final RiskLevel level;
  final List<RiskReason> reasons;
}
```

```
abstract class TelemetrySource { Stream<TelemetrySample> get samples; }
abstract class EdgeInference { Future<InferenceResult> infer(FeatureWindow w); }
abstract class RiskAssessor { RiskAssessment evaluate(...); }
abstract class ContextProvider { Future<ContextSnapshot> getContext(); }
abstract class AlertSink { Future<void> present(RiskAssessment a); }
```

The app must be able to switch TelemetrySource from BLETelemetrySource to ReplayTelemetrySource without changing monitoring logic. Replay files use the same JSON schema as BLE data and include a scenario label.

```
assets/demo/normal.json
assets/demo/heat_warning.json
assets/demo/fall_nonresponse.json
assets/demo/bad_signal.json
```

Critical engineering decision

Build the replay source on Day 1. It is both a development accelerator and your insurance policy if the sensor misbehaves in front of judges.

10-day execution plan

Work in vertical slices. Every night must end with something demoable.

15. Day-by-day plan

Day	Deliverable	Exit criteria
1	Repo scaffold + UI shell + replay telemetry	Normal/heat demo JSON can drive dashboard without hardware
2	ESP32 + MAX30102 + BME280 + MPU6050 BLE stream	Phone receives/parses packets for 15 min
3	Live dashboard + reconnect + signal quality	Can swap BLE/replay source from dev settings
4	Personal baseline + rolling window + local storage	Baseline deviations visible and persisted
5	Tiny anomaly model + TFLite integration	Fixed vector test + live inference score on phone
6	Risk engine + explanations + alert state machine	Heat replay reaches warning predictably; no alert flapping
7	NDMA SACHET adapter + ETag cache + offline mode	Cached context displayed with stale/offline labels
8	Emergency packet + fall flow + privacy/settings	Complete end-to-end demo in airplane mode
9	Qualcomm AI Hub profiling + UI polish + failure tests	Measured inference result captured; fallback modes verified
10	Code freeze + pitch rehearsal + backup devices/files	Three clean demo runs; no new features

- If BLE is not stable by end of Day 3, keep hardware display simple and lean on replay mode while one person fixes BLE.
- If the TFLite model is unstable by Day 6, use the deterministic baseline anomaly fallback behind the same interface and keep profiling as stretch.
- If NDMA integration is blocked, ship a real parser + cached sample CAP payload and label replay data clearly; do not spend two days fighting an endpoint.
- No visual redesign after Day 9 unless it fixes a demo-breaking issue.

Parallel team split

A six-person layout; combine roles if your team is smaller.

16. Workstreams

Role	Primary tasks	Dependencies
A - Firmware	Sensors, BLE payload, quality score, SOS button	Hardware
B - Mobile platform	BLE service, storage, permissions, offline state	Firmware contract
C - Mobile UX	Dashboard, alerts, history, emergency packet	Mock services on Day 1
D - ML/Edge	Feature extraction, autoencoder, TFLite export, AI Hub profile	Recorded/replay feature windows
E - Disaster/Data	NDMA SACHET parser/cache, environmental context, mock payloads	None initially
F - Integration/QA/Pitch	Risk config, test scenarios, demo controller, docs/PPT	All interfaces

1. Pull/merge only code that passes the existing replay demo.
2. Run Normal -> Heat Warning -> Bad Signal -> Fall replay scenarios.
3. Connect real hardware and compare with replay source behavior.
4. Record bugs as must-fix / nice-to-have; do not silently expand scope.
5. Update one build APK and one known-good Git tag every night.

```
main          = always demoable
feature/firmware
feature/ble
feature/ml
feature/alerts
feature/ndma
feature/ui
```

Merge through small PRs. Never let a one-day feature branch become a three-day fork.

Acceptance and failure testing

Winning demos are usually boringly reliable behind the scenes.

17. Acceptance checklist

Test	Pass condition
BLE disconnect	App enters degraded state; reconnects automatically or with one tap
Packet loss	One malformed/missing packet does not crash stream
Bad optical contact	Shows poor quality; risk is gated/suppressed
Offline mode	Inference, risk, local history and alerts still function
NDMA unavailable	Cached context shown with stale timestamp; no crash
Model asset missing	App shows explicit model error and can use dev fallback; no silent fake AI
Normal replay	No repeated notifications
Heat replay	Predictable progression through watch/warning
Fall replay	Confirmation countdown and escalation work
Delete data	Baseline/history are actually wiped
Restart app	Last baseline and recent alert history reload

Metric	Target
BLE-to-screen latency	< 2 s
Inference latency	Measure it; aim for comfortably sub-second
App startup	< 5 s on demo phone
Offline functional coverage	100% of monitoring/inference/alert path
Crash-free demo	3 consecutive full rehearsals

Replace any placeholder latency, memory, model size or accuracy number in your presentation with measured values from your actual build. Qualcomm AI Hub can profile models on physical Qualcomm devices and report runtime-related metrics; capture those results if you use it.

Judge demo script

Aim for 120 seconds. Do not begin with architecture slides.

18. Two-minute demo

Time	What to say/do
0:00-0:15	"During a heat wave, a person may not notice that their body pattern is deteriorating. Generic thresholds are the same for everyone."
0:15-0:30	Show live NORMAL dashboard and personal baseline. Point out raw health data stays local.
0:30-0:55	Start heat-risk replay/live scenario. Show risk rising and open "Why this alert?".
0:55-1:10	Turn airplane mode ON. Say: "The network is gone; the AI is not." Show inference and alerts continuing.
1:10-1:30	Show cached NDMA context / heat mode and the local recommendation + recheck/escalation state.
1:30-1:45	Trigger/advance to critical or fall non-response. Show emergency packet.
1:45-2:00	Show measured on-device inference/model size, then close with the one-line pitch.

Question	Answer direction
"Is this medically accurate?"	Prototype early warning, not diagnosis. Sensor/model require clinical validation before medical use.
"Why AI?"	The model detects multivariate pattern deviation; personalization happens relative to the user baseline; rules provide transparent safety gating.
"What happens without internet?"	BLE, local buffer, feature extraction, model, risk engine and alerts remain local. Only fresh external context is unavailable.
"Why Qualcomm?"	On-device deployment/optimization is central; AI Hub supports conversion, profiling and deployment to Qualcomm runtimes/devices.
"What makes this different?"	Personal baseline + physiological/environmental fusion + disaster-aware offline behavior, demonstrated live.

Master prompt for Codex / Claude

Attach this PDF, then paste the prompt below. Ask the agent to implement in milestones, not all at once.

19. Ready-to-paste build prompt

You are the lead engineer for the SIH26181 prototype "SwasthyaShield Edge". Treat the attached PDF as the product/architecture specification and source of truth. Build an Android-first Flutter application plus ESP32 firmware and a tiny on-device TFLite anomaly model.

Hard requirements:

1. Core monitoring, feature extraction, inference, risk scoring and alerts must work with no internet.
2. No login or backend is required for MVP. Do not upload raw health data.
3. Use a TelemetrySource abstraction with BLETelemetrySource and ReplayTelemetrySource from day one.
4. BLE packets follow the compact JSON contract in the PDF. Parser must tolerate nulls, malformed packets and disconnects.
5. Implement personal baseline statistics, rolling feature windows, EdgeInferenceService, RiskEngine and AlertCoordinator as separate modules with tests.
6. Model output is an anomaly score only. Do not implement disease diagnosis or medical treatment advice.
7. Implement NDMA SACHET through an adapter with caching. Respect ETag/If-None-Match behavior when consuming CAP XML resources. If live access is unavailable, use cached sample data but keep the parser real and label demo data.
8. Add a visible OFFLINE state. Airplane mode must not stop BLE -> features -> model -> risk -> local alert.
9. Add Normal, Heat Warning, Fall Non-response and Bad Signal replay fixtures so the complete demo is deterministic.
10. UI must show risk 0-100, level, latest vitals, signal quality, baseline deviation, disaster context timestamp and a "Why this alert?" explanation.
11. Add an emergency packet screen that summarizes recent values/trends/reasons and only includes location when consented.
12. Add Delete Local Data and Privacy screen.

Engineering constraints:

- Prefer simple maintained packages. Avoid architectural overengineering.
- Keep all thresholds/weights in one config file and label them prototype tuning parameters.
- Never fabricate model accuracy, clinical claims, disaster alerts or performance metrics.
- Create unit tests for parsers, feature ordering, risk state transitions, cache behavior and replay scenarios.
- At each milestone, provide exact run commands, files changed, tests executed and known limitations.

Implementation order:

- Milestone 1: Flutter scaffold + replay telemetry + dashboard.
- Milestone 2: ESP32 BLE stream + mobile BLE source.
- Milestone 3: baseline + feature extraction + local storage.
- Milestone 4: TFLite model training/export + local inference.
- Milestone 5: risk state machine + explanations + notifications.
- Milestone 6: NDMA SACHET cache adapter + offline/stale states.
- Milestone 7: emergency packet + fall flow + privacy controls.
- Milestone 8: integration tests + release APK + demo checklist.

Before writing code, inspect the PDF, restate the architecture in 10 bullets, create the repo tree, identify any hardware/library assumptions, and start Milestone 1. Do not ask broad product questions already answered in the PDF; make conservative engineering choices and document them.

Agent milestone prompts and deliverables

Use these when the coding agent starts drifting or trying to build too much.

20. Milestone commands

Implement only the Flutter shell, domain models, ReplayTelemetrySource and the Home dashboard.
Create four replay JSON fixtures. No BLE, no ML, no network yet.
Exit only when all four scenarios can be replayed from a hidden Dev screen.

Implement ESP32 firmware for the available sensors and compact BLE JSON notifications.
Implement BLETelemetrySource with scan/connect/reconnect and parser tests.
Do not modify risk logic; both telemetry sources must expose the same stream type.

Implement baseline -> rolling window -> ordered 12-feature vector -> TFLite inference -> risk fusion.
Add a golden test vector and verify feature order is identical in Python and Dart.
Log inference milliseconds locally. Build explanation reasons from component contributions.

Implement a provider interface and NDMA SACHET adapter. Cache CAP/RSS data locally.
Implement ETag + If-None-Match behavior for alert resource fetching and 200/304 handling.
Never block the monitoring loop on network. Include a sample CAP fixture for tests.

- [] release APK
- [] ESP32 .ino / PlatformIO project
- [] health_anomaly.tflite
- [] training/export scripts
- [] demo replay JSON files
- [] README with one-command run instructions
- [] known limitations / safety disclaimer
- [] measured model size and inference latency
- [] backup screen recording of the full demo

Verified references and safety notes

Use the official technical sources below when implementing the integrations.

21. Web references

[S1] SIH26181 problem statement mirror - GitHub repository of SIH 2026 statements.

<https://github.com/NoBugNinja/Smart-India-Hackathon-SIH-2026-Problem-Statements/blob/main/README.md> - Cross-checks ID, Qualcomm Inc, Hardware category and wording. Re-check official SIH portal before submission.

[S2] Qualcomm - On-Device AI Development. <https://www.qualcomm.com/developer/artificial-intelligence> - Official overview of frameworks, QAI RT, AI Hub and hardware-accelerated on-device inference.

[S3] Qualcomm AI Hub documentation. <https://dev.aihub.qualcomm.com/docs/> - Official conversion, validation, profiling and deployment workflow.

[S4] Qualcomm AI Hub. <https://aihub.qualcomm.com/> - Workbench supports converting custom models to on-device runtimes and profiling on hosted Qualcomm devices.

[S5] NDMA SACHET - National Disaster Alert Portal. <https://sachet.ndma.gov.in/> - Official CAP-based integrated alert system; geo-targeted multi-hazard alerts and India CAP RSS feed.

[S6] NDMA SACHET CAP XML Feed Integration Guide for Agencies.

https://sachet.ndma.gov.in/docs/Integration_Guide_For_Agencies.pdf - Official guidance for ETag caching and HTTP 200/304 handling.

[S7] Central Pollution Control Board - National AQI / AQI reports. https://cpcb.nic.in/aqi_report.php - Official AQI categories and health-impact communication. Use as contextual reference; do not mislabel non-CPCB data as CPCB AQI.

22. Safety and scientific honesty

- This hackathon prototype is not a medical device and is not intended for diagnosis, treatment, prevention or emergency dispatch.
- Consumer-grade and hobby sensors can be noisy. Demonstrate signal quality and use replay data for deterministic judging.
- All alert thresholds and fusion weights in this handoff are prototype engineering defaults, not clinical thresholds.
- Real deployment would require medical/clinical validation, calibration, regulatory review, cybersecurity threat modeling and robust human-factors testing.
- Never claim performance numbers that were not measured on your actual model/device.

Final build priority

If you finish only one exceptional thing, finish the airplane-mode demonstration: live BLE data + local anomaly inference + personalized explanation + alert, with the network visibly unavailable.